

گزارش فاز اول و دوم پروژه برنامه‌سازی وب

سامانه پخش و مدیریت موسیقی

۱. نقش اعضای گروه و فعالیت‌های انجام‌شده

پارسا حاجی قاسمی

در فاز اول بیشتر روی کامل کردن و بازبینی قسمت‌های مختلف رابط کاربری کار کردم. صفحات مدیریتی، اعلان‌ها و تنظیمات را پیاده‌سازی و اصلاح کردم، حالت واکنش‌گرای صفحات را کامل کردم و برای بخش‌های فرانت‌اند آزمون نوشتم. در فاز دوم رابط کاربری را به سرویس‌های واقعی بک‌اند متصل کردم. در این بخش روی لایه ارتباط با API، مدیریت توکن‌های احراز هویت، تبدیل ساختار داده‌های فرانت‌اند و بک‌اند، مدیریت خطاهای شبکه و اتصال صفحات به پایگاه داده کار کردم. پخش‌کننده موسیقی را هم با قابلیت‌هایی مثل صف پخش، تکرار، پخش تصادفی، انتخاب کیفیت، نوار پیشرفت و Crossfade تکمیل کردم و خطاهای مربوط به تعویض یا شروع دوباره آهنگ را برطرف کردم. در آخر، اجرای یکپارچه فرانت‌اند و بک‌اند را با Docker Compose آماده کردم و عملکرد بخش‌ها را با آزمون‌های فرانت‌اند و بک‌اند بررسی کردم.

سپهر رمضانی

سارینا فرزادنسب

۲. ساختار و پیاده‌سازی فاز اول

در فاز اول تمرکز ما روی ساخت فرانت‌اند بود. برای این کار از React، Next.js، App Router و Tailwind CSS استفاده کردیم. مسیرها را بر اساس صفحه‌های برنامه جدا کردیم و قسمت‌هایی که در چند صفحه استفاده می‌شوند، مثل نوار کناری، پوسته اصلی، کارت‌ها و پخش‌کننده را داخل پوشه components قرار دادیم. وضعیت کاربر و پخش موسیقی هم در UserContext و PlayerContext نگهداری می‌شود.

چون در شروع کار بک‌اند آماده نبود، داده‌های نمونه را در seedData.js قرار دادیم و عملیات موقت را با mockApi.js انجام دادیم. به این شکل توانستیم صفحه‌های ورود و ثبت‌نام، خانه، کتابخانه، پروفایل، پلی‌لیست، اعلان‌ها، تنظیمات و داشبوردهای هنرمند و مدیر را زودتر کامل کنیم. رابط کاربری با توجه به نقش کاربر تغییر می‌کند و صفحه‌ها را برای موبایل، تبلت و دسکتاپ طراحی کردیم.

۳. ساختار و پیاده‌سازی فاز دوم

در فاز دوم بک‌اند را با Django و Django REST Framework ساختیم و بعد آن را به فرانت‌اند وصل کردیم. برای اینکه کد بک‌اند مرتب‌تر باشد، آن را به چهار برنامه users، music، payments و support تقسیم کردیم. مدل‌ها، سریالایزرها، دسترسی‌ها، سرویس‌ها، مسیرها و آزمون‌های هر بخش در همان برنامه قرار گرفته‌اند.

احراز هویت با توکن‌های JWT انجام می‌شود و دسترسی‌ها بر اساس نقش کاربر و سطح اشتراک او کنترل می‌شوند. در فرانت‌اند، apiClient.js وظیفه ارسال درخواست، اضافه کردن توکن، تازه‌سازی توکن منقضی‌شده و یکسان کردن خطاها را دارد. فایل api.js هم داده‌های snake_case بک‌اند را به شکل مورد نیاز کامپوننت‌ها تبدیل می‌کند. در نتیجه، بعد از اتصال دو بخش، داده‌های ماک از مسیر اصلی برنامه کنار رفتند و اطلاعات از پایگاه داده و فایل‌های واقعی خوانده شدند.

طبق توضیحات README، فرانت‌اند و بک‌اند را با Docker Compose اجرا می‌کنیم. فرانت‌اند روی پورت ۳۰۰۰ و API روی پورت ۸۰۰۰ در دسترس است. داده‌ها و فایل‌های آپلودشده هم در یک حجم پایدار نگهداری می‌شوند تا با خاموش شدن کانتینرها از بین نروند.

۴. نگهداری پذیری و توسعه پذیری

برای اینکه تغییر دادن پروژه در ادامه راحت‌تر باشد، صفحه‌ها، کامپوننت‌ها، وضعیت برنامه و کدهای مربوط به ارتباط شبکه را از هم جدا کردیم. به همین دلیل تغییر ظاهر یک صفحه مستقیماً روی منطق دریافت داده اثر نمی‌گذارد. در بک‌اند هم هر حوزه را در یک برنامه جدا قرار دادیم تا فایل‌ها بیش از حد بزرگ نشوند و مسئولیت بخش‌ها با هم مخلوط نشود. قوانین مهمی مثل سطح دسترسی، محدودیت اشتراک، شمارش پخش و بررسی پرداخت را در سرور قرار دادیم تا همه کلاینت‌ها رفتار یکسانی داشته باشند.

نام‌گذاری منظم، متغیرهای محیطی، مهاجرت‌های پایگاه داده، سرویس‌های جدا، محدودیت‌های دیتابیس و آزمون‌های خودکار باعث می‌شوند اضافه کردن قابلیت جدید یا تغییر بخش‌های قبلی کم‌ریسک‌تر باشد. داکر هم کمک می‌کند همه اعضای گروه پروژه را در یک محیط مشابه و با یک فرمان مشخص اجرا کنند.

۵. مدل‌های بک‌اند و روابط آن‌ها

در بک‌اند، مدل CustomUser اطلاعات اصلی کاربر، نقش و سطح اشتراک او را نگه می‌دارد. مدل UserSettings با یک رابطه یک‌به‌یک به کاربر متصل است و تنظیمات او را ذخیره می‌کند. پروفایل Artist هم به صورت یک‌به‌یک به کاربر هنرمند وصل شده است. هر هنرمند می‌تواند چند آلبوم و آهنگ داشته باشد و هر آهنگ هم می‌تواند عضو یک آلبوم باشد یا چند هنرمند مهمان داشته باشد.

هر Playlist متعلق به یک کاربر است. برای رابطه پلی‌لیست و آهنگ از مدل واسط PlaylistTrack استفاده کردیم تا ترتیب آهنگ‌ها مشخص باشد و یک آهنگ دوبار داخل یک پلی‌لیست قرار نگیرد. هر کاربر یک PlaybackQueue دارد و آهنگ‌های مرتب‌شده صف در QueueItem ذخیره می‌شوند. مدل StreamEvent هم هر بار پخش شدن یک آهنگ توسط کاربر را ثبت می‌کند.

مدل‌های Transaction و UserSubscription تاریخچه پرداخت و دوره اشتراک کاربر را نگهداری می‌کنند و هر اشتراک پرداختی به یک تراکنش موفق وصل است. اعلان‌ها و تیکت‌ها هم به کاربر مربوط هستند و هر پاسخ تیکت به تیکت اصلی و نویسنده پاسخ متصل می‌شود. این روابط کمک می‌کنند مالک هر داده و نحوه کنترل دسترسی به آن مشخص باشد.

۶. نقش هوش مصنوعی در توسعه

در قسمت‌های مختلف پروژه برای پیدا کردن راه‌حل، رفع خطا، نوشتن آزمون، بازبینی ساختار و رعایت best practice از هوش مصنوعی کمک گرفتیم. البته کدهای پیشنهادی را مستقیماً استفاده نکردیم و قبل از اضافه کردن آن‌ها، کد را بررسی و آزمون‌ها را اجرا کردیم.

نقطه قوت هوش مصنوعی این بود که برای کارهای تکراری و پیدا کردن خطاهای احتمالی سرعت خوبی داشت. از طرف دیگر، گاهی به دلیل نداشتن شناخت کامل از پروژه یا اجرا نکردن واقعی برنامه، پیشنهاد ناقص یا خیلی عمومی می‌داد. به همین دلیل بازبینی دستی و آزمون نهایی همچنان لازم بود.

۷. جمع‌بندی

در فاز اول رابط کاربری سامانه موسیقی را ساختیم و در فاز دوم آن را به بک‌اند، پایگاه داده و احراز هویت واقعی وصل کردیم. جدا کردن مسئولیت بخش‌ها، مشخص بودن رابطه مدل‌ها، آزمون‌های خودکار و اجرای داکری باعث شده‌اند ادامه دادن و توسعه پروژه در مراحل بعدی راحت‌تر باشد.